

# Amar Dhillon

NYC-Metropolitan Area | [amardhillon844@gmail.com](mailto:amardhillon844@gmail.com) | (XXX) XXX-XXXX | [linkedin.com/in/amar-dhillon-917537261](https://linkedin.com/in/amar-dhillon-917537261) | [github.com/AmarDhillon05](https://github.com/AmarDhillon05)

## EDUCATION

**University of Maryland, College Park**  
*B.S. in Computer Science, Minor in Statistics; GPA: 3.5*

College Park, MD  
*Expected May 2028*

## CERTIFICATIONS

**Amazon Web Services (AWS) Certified Developer - Associate**  
*Amazon Web Services*

Nov 2024 – Nov 2027  
*Digital Badge*

## EXPERIENCE

**Software Development Engineer Intern**  
*Amazon Web Services*

Sep 2026 – Dec 2026  
*Redmond, WA*

- Writing backtested Java capacity-planning logic that sets upper and lower bounds on the EC2 capacity Amazon Redshift reserves for patching to balance availability against cost

**Software Engineering Intern**  
*Capital One*

Jun 2026 – Aug 2026  
*Richmond, VA*

- Cut AI workflow generation time **65%** to under 15s by splitting a Claude skill into focused sub-skills that call Python search and validation tools instead of searching open-ended
- Precomputed popular workflows from a 270K+ row PostgreSQL table into a Redis-cached category tree to keep lookups within the database's permission and rate limits, which became the Claude skill's main search tool
- Rebuilt a legacy test-data workflow tool as a React and TypeScript Electron app with a visual React Flow editor and Jest-tested validation to make editing smoother, which **90%** of 30+ surveyed developers preferred

**Software Engineer (Contract)**  
*UMD App Development Contracting (Client: Amazon Leo)*

Jan 2026 – Present  
*College Park, MD*

- Cut end-to-end latency **34%** on a data pipeline that sends 6,000+ scraped research documents to model inference by streaming them through Amazon Kinesis into AWS Lambda instead of staging each step in S3
- Co-built a research tool that scores and aggregates topics to show Amazon Leo's public-policy staff what's popular, which runs in dev on AWS Fargate with GitHub Actions CI/CD and AWS CDK infrastructure-as-code ready for production
- Built an EventBridge-scheduled LLM-as-judge check for a production Amazon Bedrock classifier that rates research documents by importance to trigger fine-tuning only when its grades shift past a threshold

**Machine Learning Research Intern**  
*UMD Institute for Advanced Computer Studies (UMIACS)*

Dec 2025 – Present  
*College Park, MD*

- Cut research-environment setup time **30%** by pinning dependencies with uv and packaging the environment in Docker so every machine gets an identical setup

**Co-Founder, Software Engineer**  
*TerpLabs*

Jun 2025 – Present  
*College Park, MD*

- Built Redis caching in front of the Prisma ORM queries for Tortuga, the studio's class scheduler with 20,000+ visits in its first 30 days, to reduce database load and latency

## PROJECTS

**IEX Order Book & Matching Engine** | *C++17, STL, POSIX, Linux perf*

Jul 2026 – Aug 2026

- Built a C++17 order book and matching engine to replay **50.9M** real IEX exchange messages without ever resyncing, which agreed with the exchange on 99,122 of 99,123 fills and 100% of top-of-book prices
- Implemented B+ tree and LSM-tree backends for the engine's event log to compare them on identical 2M-record workloads, which showed the B+ tree giving **12.8x** the read throughput and 22x faster range queries for 27% more disk
- Designed the book as std::list FIFOs per price level inside a std::map with an order-ID hash map to make cancels O(1), which sustained **1.26M events/sec** end-to-end over the full replay

## TECHNICAL SKILLS

**Languages:** Java, Python, C++, TypeScript, JavaScript, SQL, Bash

**Cloud & Infrastructure:** AWS (EC2, Lambda, Kinesis, Fargate/ECS, API Gateway, EventBridge, SNS), observability (CloudWatch, X-Ray), AWS CDK (infrastructure as code), Docker, GitHub Actions (CI/CD), Linux

**Backend, Full-Stack & Data:** PostgreSQL, Redis, SQLAlchemy, Prisma, OpenSearch, REST APIs, Node.js, B+ tree and LSM-tree storage

**Practices & Tools:** debugging and performance profiling (perf, psutil), benchmarking, unit testing (Pytest, Jest), data structures and algorithms, Git, uv